

Module 1

Design Patterns and Principles

Self-Learning Report

I completed Module 1 on Design Patterns and Principles, building a working understanding of core software design concepts and industry best practices. Through this module, I came to appreciate why software needs to be built for maintainability, scalability, reusability, and extensibility, and how sound design principles within Object-Oriented Programming (OOP) support the creation of dependable, effective software systems.

The module centered on the SOLID principles of object-oriented design, giving me a clear picture of what each principle means and where it applies. The key topics I explored were:

- **Single Responsibility Principle (SRP):** A class should be built around a single responsibility, giving it just one reason to change.
- **Open-Closed Principle (OCP):** Software entities should be designed so new behavior can be added through extension, without altering existing, tested code.
- **Liskov Substitution Principle (LSP):** Objects of a subclass should be usable wherever the parent class is expected, without breaking the program's correctness.
- **Interface Segregation Principle (ISP):** It's better to have several small, focused interfaces than one large, general-purpose interface that forces classes to implement methods they don't need.
- **Dependency Inversion Principle (DIP):** High-level modules should rely on abstractions rather than concrete implementations, reducing tight coupling between components.

Alongside SOLID, I studied several well-established Gang of Four (GoF) design patterns, which offer reusable, proven solutions to recurring design challenges. The patterns and architectural styles I covered include:

- **Creational Patterns:** Singleton, Factory Method, and Builder — patterns focused on controlled and flexible object creation.
- **Structural Patterns:** Adapter, Decorator, and Proxy — patterns that define how classes and objects are composed into larger structures.
- **Behavioral Patterns:** Observer, Strategy, and Command — patterns concerned with communication and responsibility-sharing between objects.
- **Architectural Patterns:** Model-View-Controller (MVC) and Dependency Injection, which shape how larger applications are structured and how components are wired together.

Working through hands-on exercises and practical code examples helped me learn to spot common design problems and choose the right pattern to address them. This module strengthened my understanding of how good design principles and patterns lead to more reusable, maintainable code with less duplication, and how they support building software architectures that scale well over time. Overall, Module 1 deepened my grasp of object-oriented design and gave me practical experience in crafting clean, efficient software solutions.